PROYECTO INTEGRADOR

TECHCART

Tienda web de productos tecnológicos

Carrito de compras · Programación Orientada a Objetos · SPA · Responsive

Presentación técnica y funcional del proyecto



HTML5 · CSS3 · JavaScript · LocalStorage · JSON

Presentación del proyecto

Una versión más clara y enfocada del flujo de presentación usado como referencia.

01 **Problema y oportunidad**

02 **Objetivo y alcance**

03 **Requisitos funcionales**

04 **Arquitectura y POO**

05 **Interfaz y experiencia**

06 **Responsive y cierre**

TechCart toma la estructura temática de la presentación de referencia - problema, objetivo, alcance, requisitos, arquitectura, interfaz y responsive - pero la adapta a una tienda tecnológica y la simplifica para una exposición más efectiva.

Descripción del problema

Comprar tecnología en línea suele implicar catálogos dispersos, procesos de compra poco claros y escasa continuidad entre navegación, carrito y orden.

El reto

Centralizar el catálogo tecnológico en una experiencia coherente.
Permitir registro e inicio de sesión con validación.
Mantener el carrito y los datos del cliente entre vistas.
Actualizar la interfaz sin recargar la aplicación.

Navegación

Flujo claro desde la landing hasta catálogo, carrito y orden.

Acceso

Registro, login, tres intentos y modificación segura del perfil.

Compra

Agregar, modificar cantidades, eliminar y recalcular total.

Responsive

Uso consistente en escritorio, tablet y móvil.

Análisis de la solución

TechCart responde al problema con una arquitectura simple, modular y demostrable en el navegador.

01 **Experiencia pública**

Landing, búsqueda, categorías y llamadas a login/registro.

02 **Experiencia privada**

SPA con sidebar, catálogo, carrito, compras y perfil.

03 **Persistencia simulada**

LocalStorage para clientes, sesión y carrito; JSON para catálogo.

04 **POO real**

Producto como superclase y subclases Laptop, Smartphone y Accesorio.

Resultado esperado: una simulación de comercio electrónico funcional, presentable y alineada con los criterios de evaluación.

Objetivo del proyecto

Desarrollar una aplicación web funcional que simule una tienda de productos tecnológicos y un carrito de compras, aplicando Programación Orientada a Objetos, manipulación del DOM, persistencia simulada y diseño responsive.

El proyecto prioriza claridad, modularidad, experiencia de usuario y evidencia explícita de herencia y polimorfismo.

Alcance - sí incluye

Gestión de clientes

Registro, login, máximo tres intentos y modificación de datos con verificación de contraseña.

Catálogo

Productos tecnológicos cargados desde JSON y convertidos a objetos de dominio.

Carrito

Agregar, eliminar, cambiar cantidades y total en tiempo real.

Orden

Generación de resumen final con productos, total, fecha y estado.

SPA

Panel interno sin recarga, con navegación por vistas y sidebar responsive.

TechBot

Chat flotante con respuestas predeterminadas y navegación guiada.

Persistencia

LocalStorage para sesión, clientes y carrito; catálogo externo en JSON.

Responsive

Desktop, tablet y móvil.

Alcance - no incluye

La solución es una simulación académica orientada al frontend y a la lógica de negocio en JavaScript.

Pagos reales

No integra pasarelas de pago ni procesa dinero real.

Base de datos remota

No usa MySQL, PostgreSQL ni servicios cloud de producción.

Logística real

No conecta transportadoras ni rastreo de envíos real.

Seguridad productiva

Las credenciales simuladas se almacenan localmente; no reemplaza autenticación de servidor.

Misión y visión

MISIÓN

Ofrecer una experiencia de compra tecnológica clara, intuitiva y confiable, integrando catálogo, carrito, gestión del cliente y orden de compra en una sola interfaz web.

VISIÓN

Evolucionar TechCart hacia una plataforma modular y escalable que permita ampliar categorías, servicios y persistencia sin rehacer la arquitectura central.

Requisitos funcionales - gestión de clientes

01 **Registro**

Crear cuenta con nombre, usuario, correo, dirección y contraseña.

02 **Inicio de sesión**

Validar credenciales y limitar a tres intentos fallidos.

03 **Sesión**

Mantener el usuario autenticado y proteger el panel interno.

04 **Modificación**

Bloquear datos hasta verificar contraseña actual; permitir cambio de información y clave.

Requisitos funcionales - productos y carrito

Catálogo

Nombre, imagen, descripción, especificaciones y precio.

Agregar

Añadir productos y mantener cantidad por referencia.

Modificar

Aumentar o disminuir unidades en tiempo real.

Eliminar

Retirar productos del carrito.

Total

Recalcular valor total de forma inmediata.

Confirmar

Crear una orden con detalle, fecha, total y estado.

Flujo principal del usuario



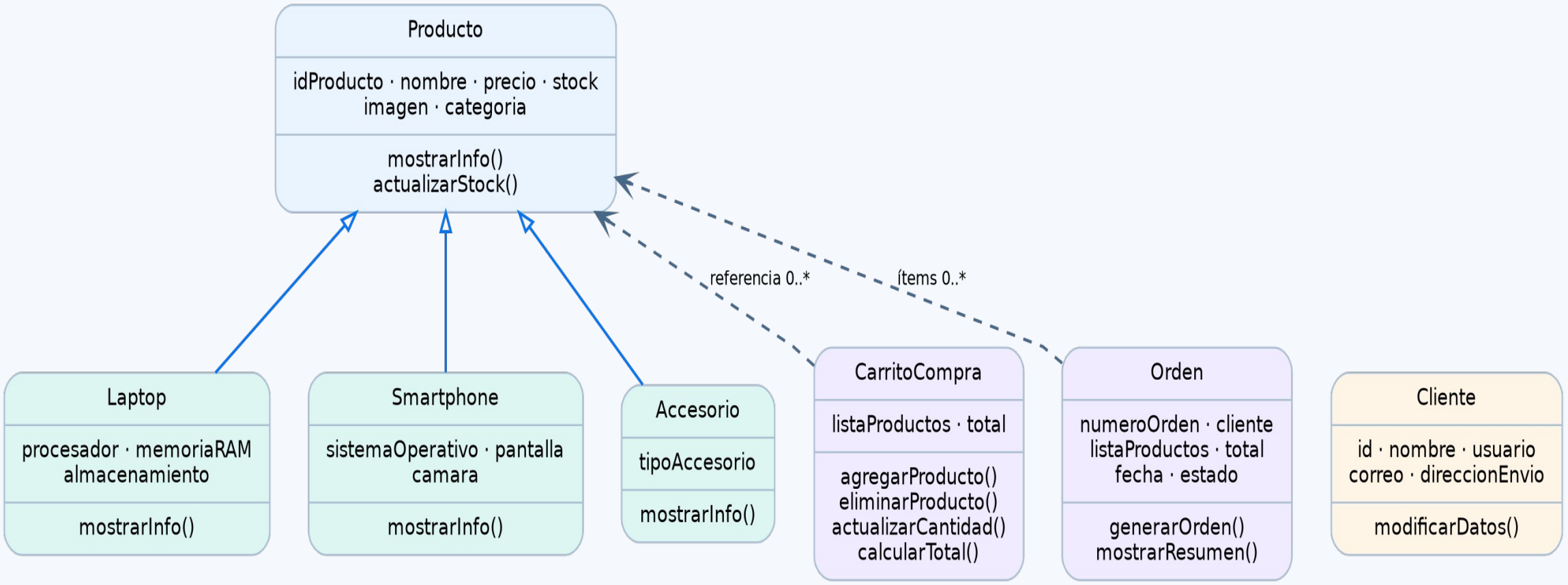
La navegación interna funciona como SPA: el Router cambia la vista activa sin recargar el documento y los módulos mantienen responsabilidades separadas.

Arquitectura modular

Interfaz / Views	PanelApp · Router · Sidebar · ProfileView · ProductsView · CartView · OrdersView · Chatbot
Servicios	AuthService · ClienteService · ProductService · StorageService
Modelos POO	Cliente · Producto · Laptop · Smartphone · Accesorio · CarritoCompra · Orden
Persistencia	LocalStorage + productos.json

UML DE CLASES · HERENCIA CORRECTA

Solo Laptop, Smartphone y Accesorio heredan de Producto. Otras líneas representan referencias o dependencias.



△ **Triángulo hueco → generalización hacia superclase** **• • • → referencia/dependencia (sin herencia)**

Cómo se evidencian los principios de POO

Herencia

Laptop, Smartphone y Accesorio reutilizan atributos y métodos definidos en Producto.

Polimorfismo

Cada subclase sobrescribe mostrarInfo() para exponer especificaciones particulares.

Encapsulación conceptual

Cada clase concentra datos y operaciones relacionadas con una responsabilidad del dominio.

Separación

Modelos, servicios y vistas se dividen en archivos distintos para reducir acoplamiento.

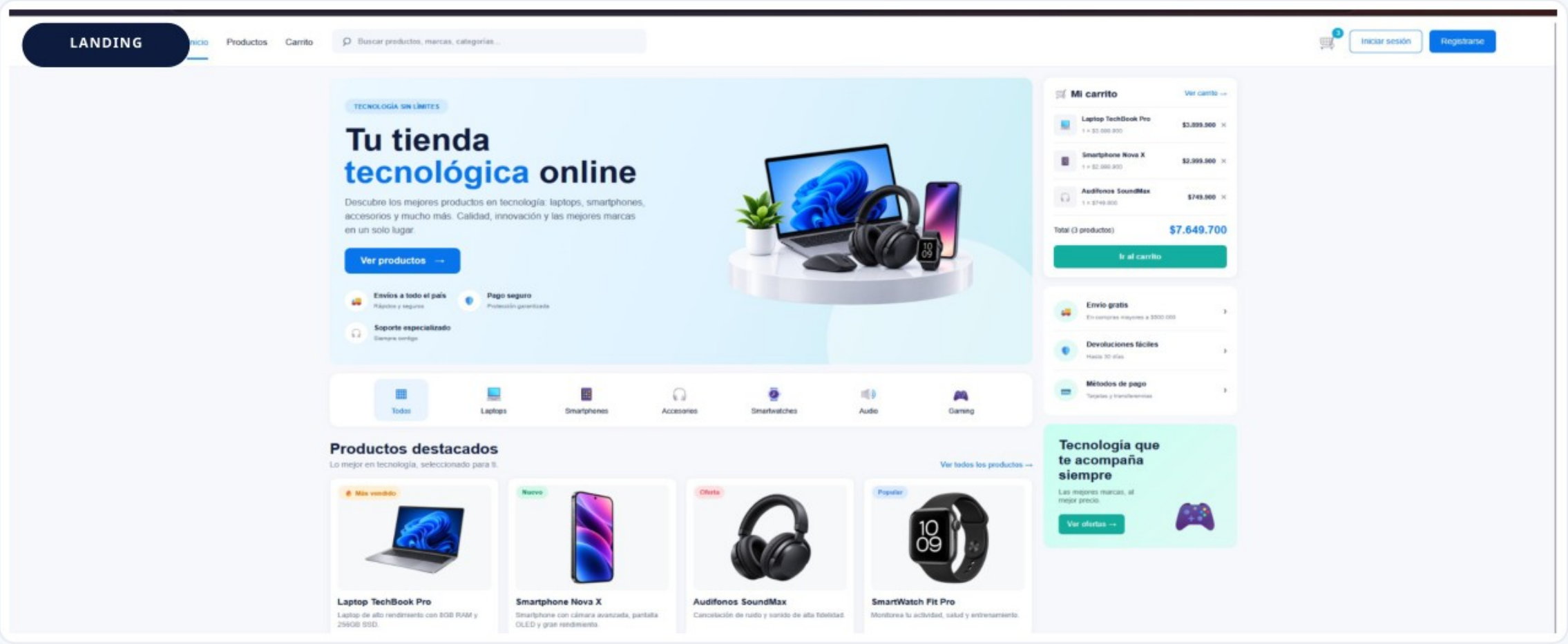
Ejemplo de polimorfismo: ProductsView llama producto.mostrarInfo() sin necesitar saber si el objeto concreto es Laptop, Smartphone o Accesorio.

Datos y persistencia simulada



Ventaja: agregar nuevos productos se reduce a editar el JSON y asociar su imagen; la interfaz se genera dinámicamente con DOM.

Interfaz pública - landing



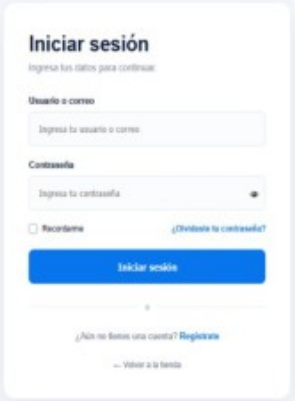
Página de entrada con catálogo destacado, categorías, buscador, carrito visual y llamadas a iniciar sesión o registrarse.

Acceso y autenticación

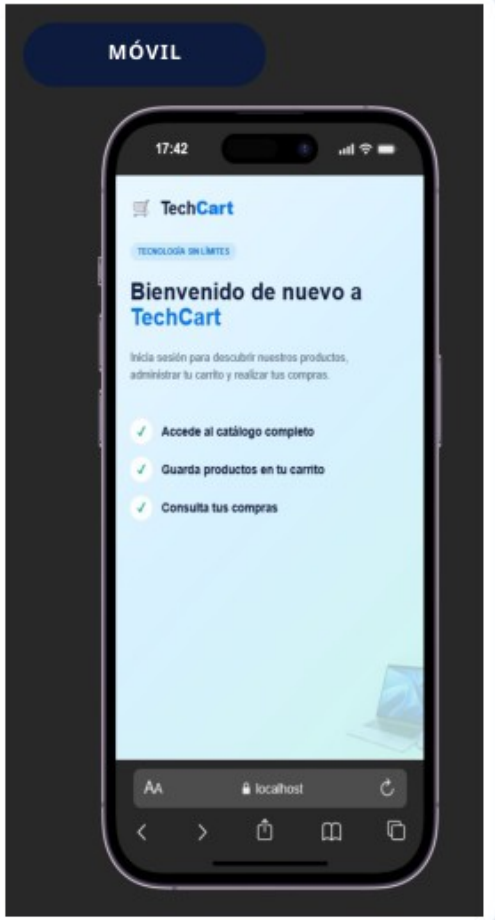
El formulario guía al usuario y mantiene una identidad visual consistente con la tienda.

DESKTOP



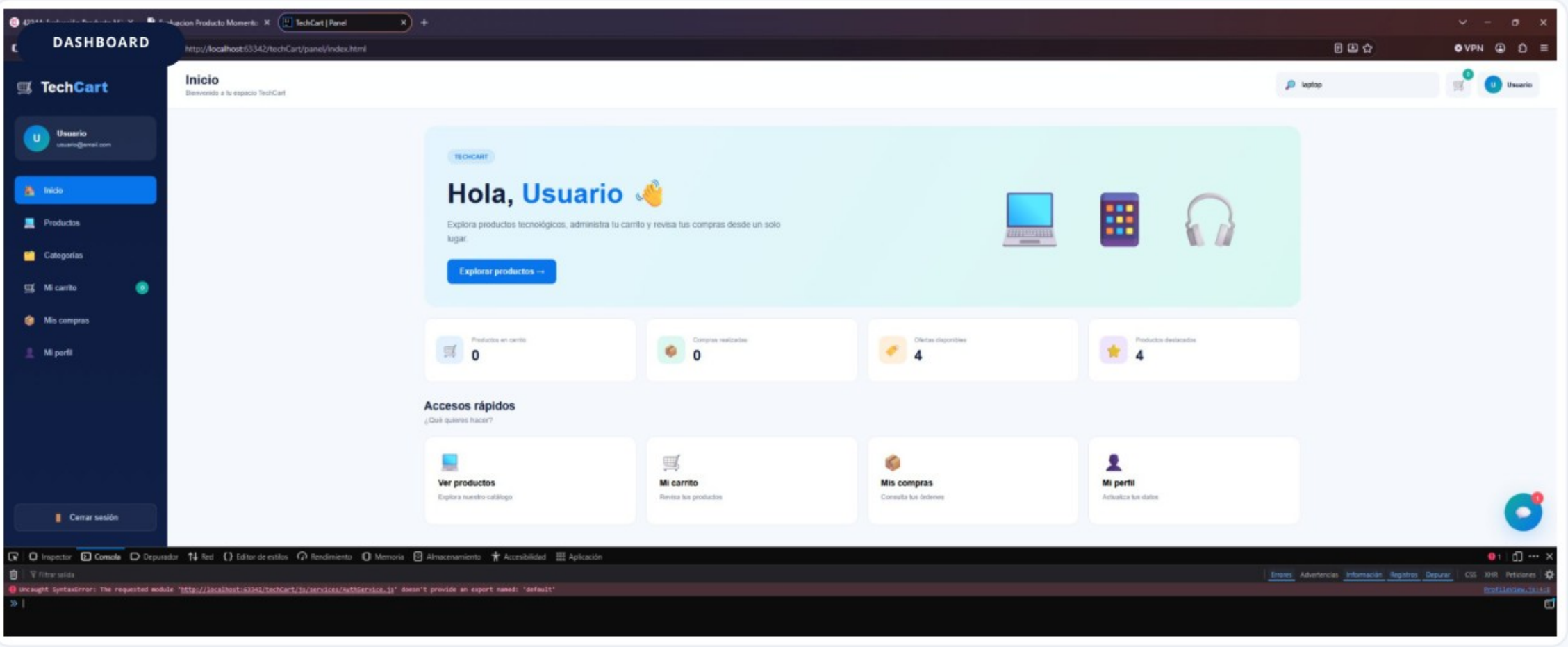


MÓVIL



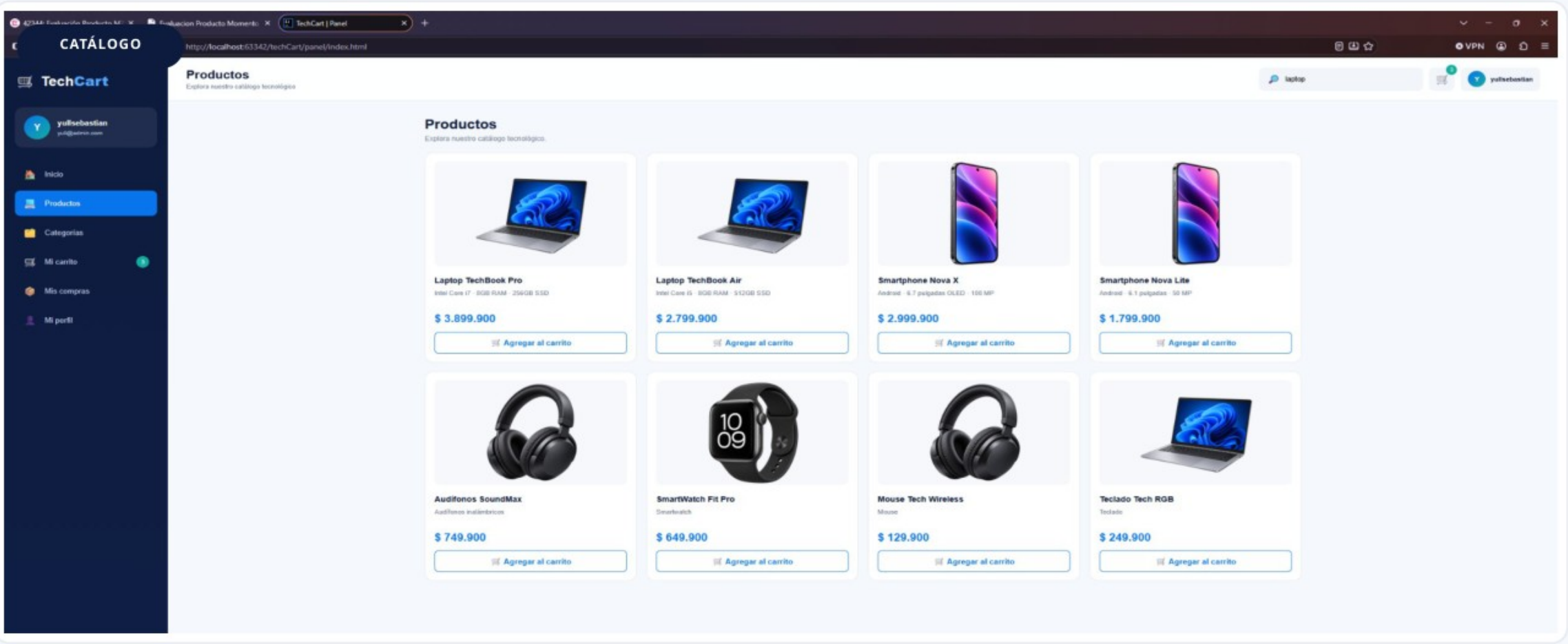
Panel interno SPA

Una sola página, varias vistas y navegación sin recarga.



Catálogo dinámico

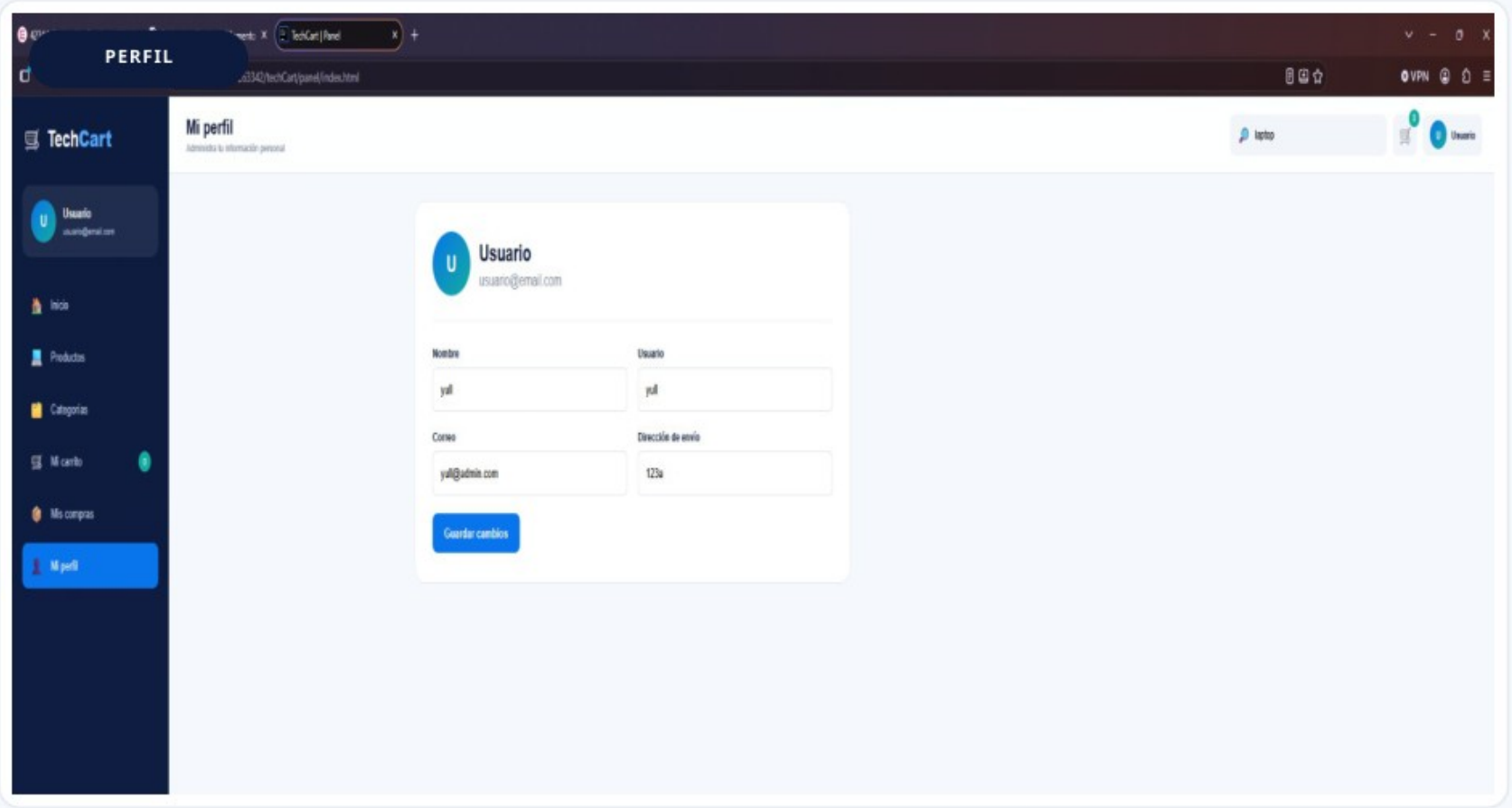
Los productos se renderizan desde objetos creados a partir de productos.json.



El contador del carrito se actualiza y la información permanece en LocalStorage.

Perfil y seguridad de cambios

Los datos del cliente se sincronizan con la sesión y solo se editan después de verificar la contraseña actual.



Modo lectura

Campos bloqueados hasta pulsar “Cambiar datos”.

Verificación

SweetAlert2 solicita la contraseña actual antes de habilitar edición.

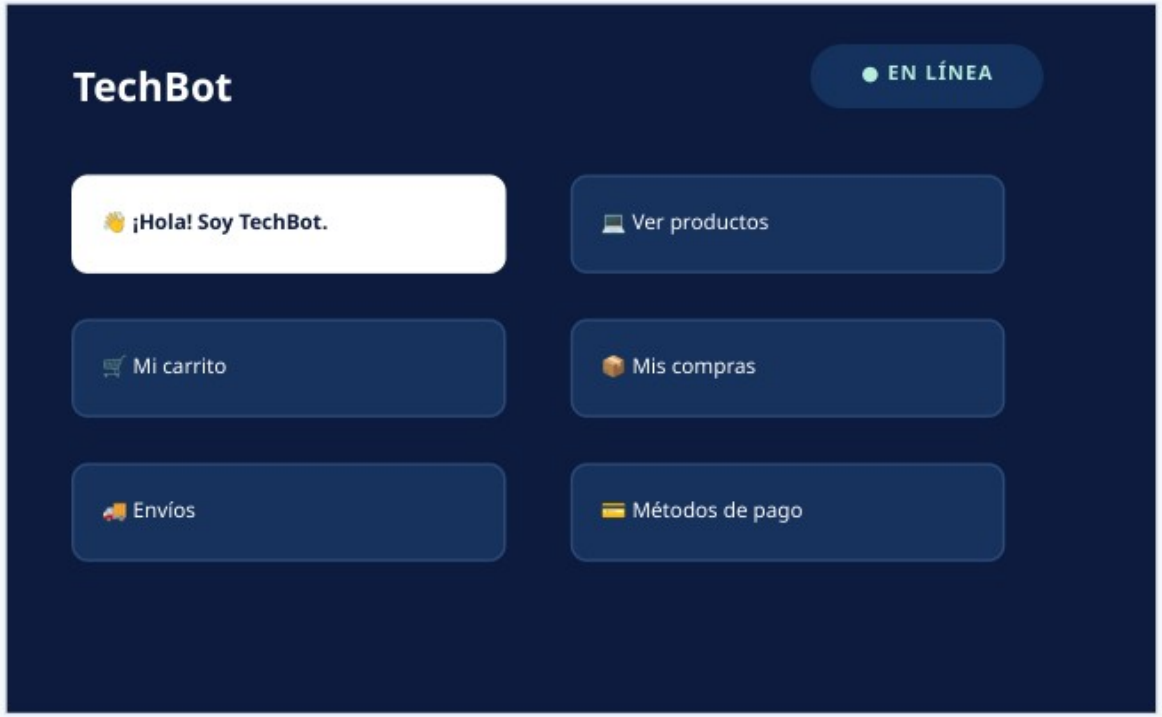
Cambio de clave

La nueva contraseña se valida y se confirma antes de guardar.

TechBot y navegación guiada

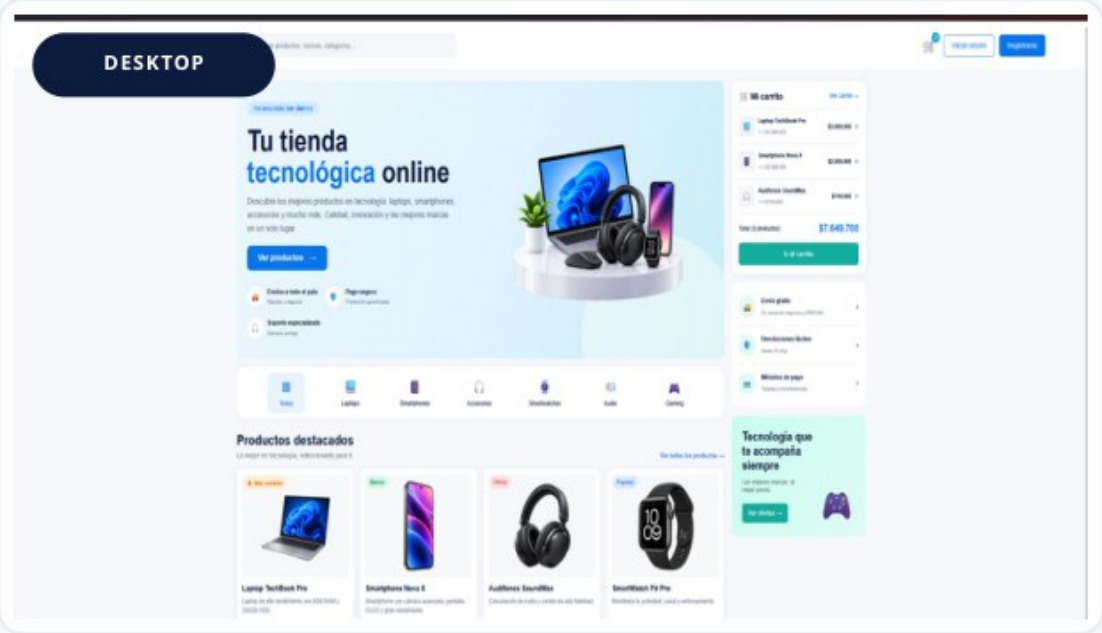
Chat flotante

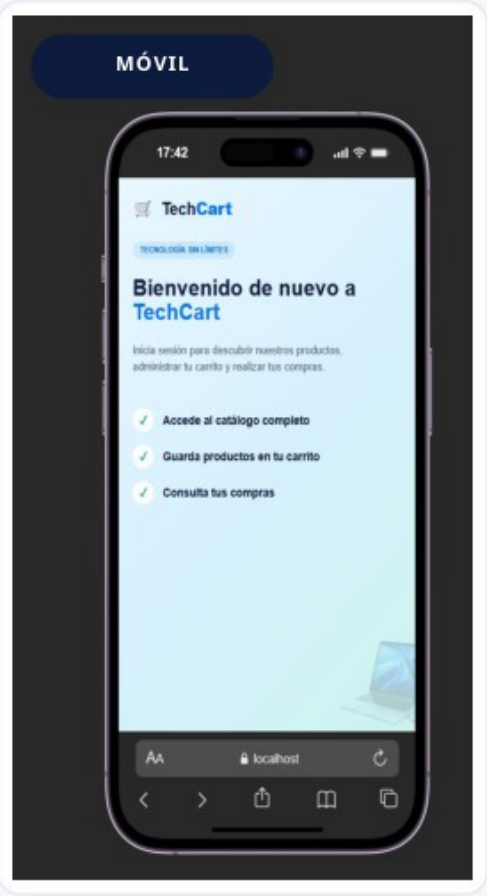
Botón fijo en la esquina inferior derecha.
Respuestas predeterminadas para productos, carrito, compras, envíos, pagos y perfil.
Puede activar vistas del SPA mediante Router.
Se adapta al ancho disponible en móvil.



Diseño responsive

La evaluación exige adaptación a escritorio, tablet y móvil; TechCart aplica breakpoints en landing, autenticación y panel.

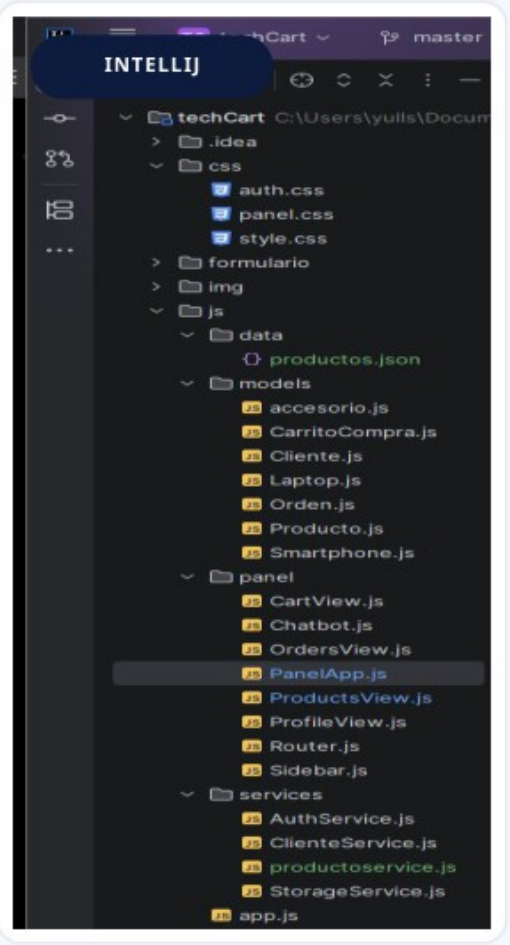




Comportamiento

Sidebar colapsable.
Grid de productos adaptable.
Formulario en una columna.
Chat a ancho móvil.
Sin desbordamiento horizontal.

Estructura del proyecto y responsabilidades



models/ Entidades y reglas del dominio: Cliente, Producto, subclases, CarritoCompra y Orden.

services/ Persistencia, autenticación, cliente y carga del catálogo.

panel/ Vistas SPA, Router, Sidebar, Chatbot y orquestación.

data/ productos.json como fuente de catálogo.

Principio aplicado: una responsabilidad principal por módulo.

Alineación con la lista de chequeo

- ✓ Diseño moderno y navegación clara
- ✓ Responsive desktop / tablet / móvil
- ✓ Registro, login y máximo tres intentos
- ✓ Modificación segura de datos del cliente
- ✓ Catálogo con imagen, descripción y precio
- ✓ Agregar / modificar / eliminar del carrito
- ✓ Total actualizado en tiempo real
- ✓ Clases Producto, Cliente, CarritoCompra y Orden
- ✓ Herencia: Laptop, Smartphone y Accesorio
- ✓ Polimorfismo mediante mostrarInfo()

TECHCART

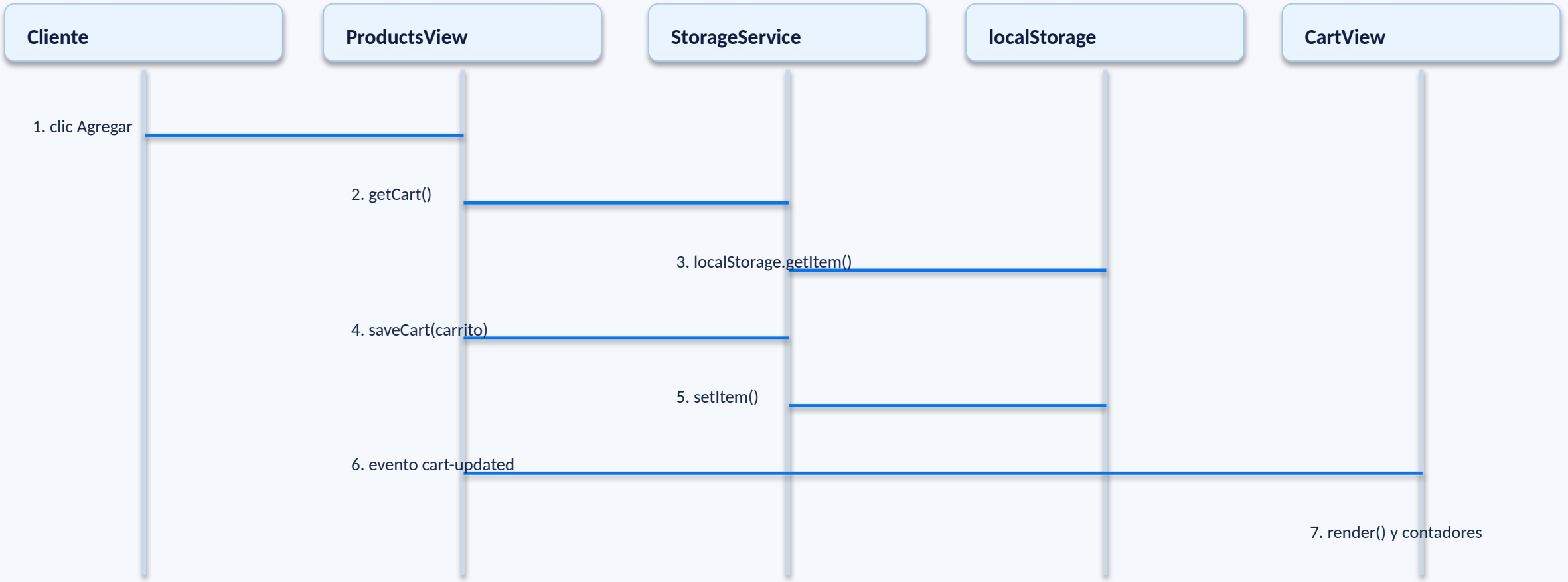
MUCHAS GRACIAS

Una tienda tecnológica construida para demostrar POO, DOM, persistencia simulada y diseño responsive.



SECUENCIA REAL · AGREGAR AL CARRITO

Alineada con ProductsView.js y StorageService.js revisados; no se inventa una llamada directa a CarritoCompra.



Nota: ProductsView agrega al arreglo y persiste con StorageService; CartView reacciona al evento.

AMPLIACIÓN DEL ALCANCE

Funcionalidades añadidas después del MVP original; todas operan en frontend con datos locales o simulados.

72 productos y catálogo

Ampliación de 18 a 72 artículos; búsqueda por texto, categorías y tarjetas con detalles.

Comparador

Selección de hasta 3 productos; filtros por tipo, tabla de atributos y agregar al carrito.

Reseñas y calificaciones

Estrellas 1–5, comentarios, promedios por producto y almacenamiento local.

Detalle de producto

Modal con imagen, características, stock, precio y acciones de compra.

Simulador de envíos

Destino, modalidad, tarifa y estimación; desglose asociado a la orden y PDF.

TechBot asíncrono

Consultas de envíos y pagos de prueba con Promesas y async/await; sin cobros reales.

HISTORIAS DE USUARIO NUEVAS · HU-11 A HU-15

Continuación del backlog original HU-01 a HU-10. Historias redactadas según funcionalidades observadas o descritas.

HU-11	**Comparar productos**	Como cliente, quiero comparar hasta tres productos de la misma familia para escoger con mejores criterios.
HU-12	**Reseñas**	Como cliente, quiero puntuar y comentar productos para compartir mi valoración y consultar promedios.
HU-13	**Ficha detallada**	Como cliente, quiero abrir una ficha con datos y acciones del producto sin perder la navegación.
HU-14	**Costo de envío**	Como cliente, quiero estimar tarifa y modalidad de envío y ver el total antes de confirmar.
HU-15	**TechBot asíncrono**	Como cliente, quiero consultar envíos y explorar medios de pago simulados sin bloquear la pantalla.

CRITERIOS DE ACEPTACIÓN · NUEVAS FUNCIONES

Criterios verificables para la demostración académica; validación integrada pendiente de pruebas del entorno final.

HU-11 · Comparador

Permite categoría y búsqueda; selecciona máximo 3; compara especificaciones compatibles; agrega la opción elegida al carrito.

HU-12 · Reseñas

Permite estrellas 1–5 y comentario; calcula promedio; actualiza en catálogo; almacena datos en navegador.

HU-13 · Detalles

Abre modal desde tarjeta; muestra descripción, especificaciones, stock y precio; permite comparar y agregar.

HU-14/15 · Envíos y bot

Cotiza según destino/modalidad; guarda desglose en orden y PDF; muestra pagos como simulación sin datos bancarios reales.

ARQUITECTURA MODULAR ACTUALIZADA

Separación de interfaz, servicios, modelos y persistencia. La asincronía no implica backend ni validación bancaria.

VISTAS / UI

PanelApp, Router, Sidebar
ProductsView, ProductDetailView
ComparadorView, ReviewsView
CartView, OrdersView, HistoryView
ProfileView, Chatbot, GlobalSearch

SERVICIOS

ProductService
ComparadorService
ResenaService
EnvioService
StorageService
AuthService, ClienteService

MODELOS / DATOS

Producto; Laptop; Smartphone
Accesorio; Cliente
CarritoCompra; Orden
ComparadorProductos; Resena
productos.json; localStorage

Regla UML: importación/uso ≠ herencia. La herencia JavaScript confirmada es Producto ← Laptop/Smartphone/Accesorio.

ASINCRONÍA APLICADA AL NEGOCIO

TechBot usa un flujo de consulta simulada; la UI continúa respondiendo mientras espera temporizadores y Promesas.

1 · Evento del usuario

Clic en Métodos de pago o Envíos

2 · Call Stack

Se inicia la función async; se presenta estado pendiente

3 · Web API / temporizador

setTimeout simula latencia; no consulta servicios bancarios

4 · Task Queue

Callback del temporizador se ejecuta cuando toca

5 · Microtask Queue

Se reanuda await al resolverse la Promesa

6 · Render

Se muestra estimación o estado de simulación

LIMITACIONES Y CONDICIONES DE DEMOSTRACIÓN

Precisiones necesarias al presentar TechCart en clase.

Qué SÍ se demuestra

POO: herencia y sobrescritura.
SPA y manipulación del DOM.
Persistencia local y órdenes PDF.
Búsqueda con debounce.
Promesas, async/await y Event Loop.
Tarifas y tiempos de envío simulados.

Qué NO debe afirmarse

No valida bancos ni descuenta dinero real.
No verifica inventario compartido.
No hay pasarela bancaria ni API logística.
localStorage no es una base remota.
Autenticación local no es seguridad productiva.

Evidencia de actualización: código compartido, capturas de la SPA y descripción del flujo final; no se ha auditado todo el repositorio.